Final_project

2024-11-17

Exploratory Data Analysis

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-8
```

```r
library(rstan)
```

```
## Loading required package: StanHeaders
##
## rstan version 2.32.6 (Stan version 2.32.2)
##
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
## change `threads_per_chain` option:
## rstan_options(threads_per_chain = 1)
##
##
## Attaching package: 'rstan'
##
## The following object is masked from 'package:tidyr':
##
```

```
##     extract
```

```r
library(bayesplot)
```

```
## This is bayesplot version 1.11.1
## - Online documentation and vignettes at mc-stan.org/bayesplot
## - bayesplot theme set to bayesplot::theme_default()
##    * Does _not_ affect other ggplot2 plots
##    * See ?bayesplot_theme_set for details on theme setting
```

```r
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
data <- read.csv("/Users/yiouwang/Desktop/h1n1_vaccine_prediction.csv")
data[data == ""] <- NA
categorical_vars <- c("age_bracket", "qualification", "race", "sex",
                      "income_level", "marital_status", "housing_status",
                      "employment", "census_msa")
summary(data)
```

```
##    unique_id       h1n1_worry    h1n1_awareness  antiviral_medication
##  Min.   :    0   Min.   :0.000   Min.   :0.000   Min.   :0.00000
##  1st Qu.: 6676   1st Qu.:1.000   1st Qu.:1.000   1st Qu.:0.00000
##  Median :13353   Median :2.000   Median :1.000   Median :0.00000
##  Mean   :13353   Mean   :1.618   Mean   :1.263   Mean   :0.04884
##  3rd Qu.:20030   3rd Qu.:2.000   3rd Qu.:2.000   3rd Qu.:0.00000
##  Max.   :26706   Max.   :3.000   Max.   :2.000   Max.   :1.00000
##                  NA's   :92      NA's   :116     NA's   :71
##  contact_avoidance bought_face_mask  wash_hands_frequently
##  Min.   :0.0000    Min.   :0.00000   Min.   :0.0000
##  1st Qu.:0.0000    1st Qu.:0.00000   1st Qu.:1.0000
##  Median :1.0000    Median :0.00000   Median :1.0000
##  Mean   :0.7256    Mean   :0.06898   Mean   :0.8256
##  3rd Qu.:1.0000    3rd Qu.:0.00000   3rd Qu.:1.0000
##  Max.   :1.0000    Max.   :1.00000   Max.   :1.0000
##  NA's   :208       NA's   :19        NA's   :42
##  avoid_large_gatherings reduced_outside_home_cont avoid_touch_face
##  Min.   :0.0000         Min.   :0.0000            Min.   :0.0000
##  1st Qu.:0.0000         1st Qu.:0.0000            1st Qu.:0.0000
##  Median :0.0000         Median :0.0000            Median :1.0000
##  Mean   :0.3586         Mean   :0.3373            Mean   :0.6773
```

```
##  3rd Qu.:1.0000         3rd Qu.:1.0000            3rd Qu.:1.0000
##  Max.   :1.0000         Max.   :1.0000            Max.   :1.0000
##  NA's   :87             NA's   :82                NA's   :128
##  dr_recc_h1n1_vacc dr_recc_seasonal_vacc chronic_medic_condition
##  Min.   :0.0000    Min.   :0.0000        Min.   :0.0000
##  1st Qu.:0.0000    1st Qu.:0.0000        1st Qu.:0.0000
##  Median :0.0000    Median :0.0000        Median :0.0000
##  Mean   :0.2203    Mean   :0.3297        Mean   :0.2833
##  3rd Qu.:0.0000    3rd Qu.:1.0000        3rd Qu.:1.0000
##  Max.   :1.0000    Max.   :1.0000        Max.   :1.0000
##  NA's   :2160      NA's   :2160          NA's   :971
##  cont_child_undr_6_mnths is_health_worker has_health_insur
##  Min.   :0.0000          Min.   :0.0000   Min.   :0.00
##  1st Qu.:0.0000          1st Qu.:0.0000   1st Qu.:1.00
##  Median :0.0000          Median :0.0000   Median :1.00
##  Mean   :0.0826          Mean   :0.1119   Mean   :0.88
##  3rd Qu.:0.0000          3rd Qu.:0.0000   3rd Qu.:1.00
##  Max.   :1.0000          Max.   :1.0000   Max.   :1.00
##  NA's   :820             NA's   :804      NA's   :12274
##  is_h1n1_vacc_effective is_h1n1_risky   sick_from_h1n1_vacc
##  Min.   :1.000          Min.   :1.000   Min.   :1.000
##  1st Qu.:3.000          1st Qu.:1.000   1st Qu.:1.000
##  Median :4.000          Median :2.000   Median :2.000
##  Mean   :3.851          Mean   :2.343   Mean   :2.358
##  3rd Qu.:5.000          3rd Qu.:4.000   3rd Qu.:4.000
##  Max.   :5.000          Max.   :5.000   Max.   :5.000
##  NA's   :391            NA's   :388     NA's   :395
##  is_seas_vacc_effective is_seas_risky   sick_from_seas_vacc age_bracket
##  Min.   :1.000          Min.   :1.000   Min.   :1.000       Length:26707
##  1st Qu.:4.000          1st Qu.:2.000   1st Qu.:1.000       Class :character
##  Median :4.000          Median :2.000   Median :2.000       Mode  :character
##  Mean   :4.026          Mean   :2.719   Mean   :2.118
##  3rd Qu.:5.000          3rd Qu.:4.000   3rd Qu.:4.000
##  Max.   :5.000          Max.   :5.000   Max.   :5.000
##  NA's   :462            NA's   :514     NA's   :537
##  qualification          race               sex            income_level
##  Length:26707       Length:26707       Length:26707       Length:26707
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##  marital_status     housing_status      employment         census_msa
##  Length:26707       Length:26707       Length:26707       Length:26707
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
##   no_of_adults    no_of_children    h1n1_vaccine
##  Min.   :0.0000   Min.   :0.0000   Min.   :0.0000
##  1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.0000
```

```
##  Median :1.0000   Median :0.0000   Median :0.0000
##  Mean   :0.8865   Mean   :0.5346   Mean   :0.2125
##  3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:0.0000
##  Max.   :3.0000   Max.   :3.0000   Max.   :1.0000
##  NA's   :249      NA's   :249
```

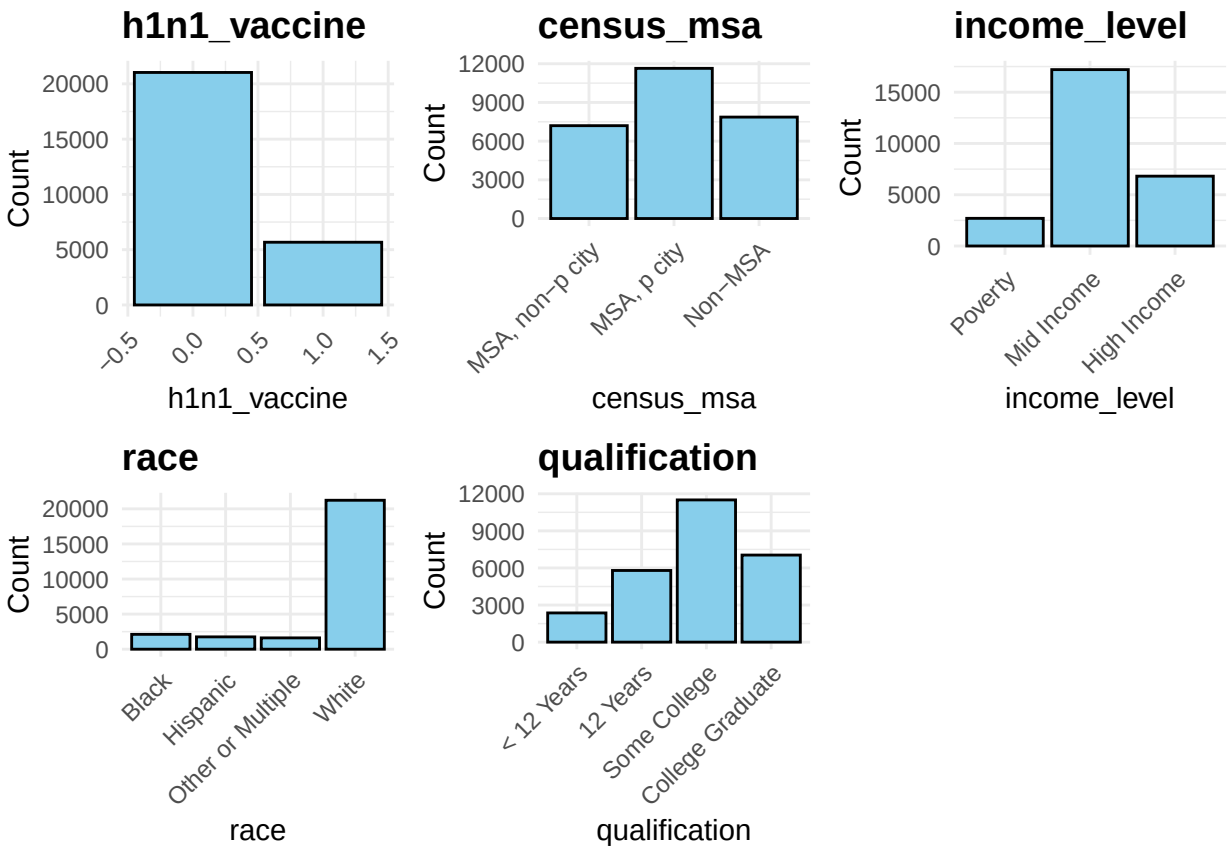
```
mode_func <- function(x) {
  uniq_x <- unique(x)
  uniq_x[which.max(tabulate(match(x, uniq_x)))]
}

# Missing values in numeric columns were replaced with their median
# Missing values in categorical columns were replaced with the mode, using a custom mode_func function
data_imputed <- data %>%
  mutate(across(where(is.numeric), ~ ifelse(is.na(.), median(., na.rm = TRUE), .))) %>%
  mutate(across(all_of(categorical_vars), ~ ifelse(is.na(.), mode_func(.), .)))


data_encoded <- data_imputed %>%
  mutate(
    income_level = as.numeric(factor(income_level,
                                     levels = c("Below Poverty", "<= $75,000, Above Poverty", "> $75,000
                                     ordered = TRUE)),
    age_bracket = as.numeric(factor(age_bracket,
                                    levels = c("18 - 34 Years", "35 - 44 Years", "45 - 54 Years", "55 -
                                    ordered = TRUE)),
    qualification = as.numeric(factor(qualification,
                                      levels = c("< 12 Years", "12 Years", "Some College", "College Grad
                                      ordered = TRUE)),
    census_msa = as.numeric(factor(census_msa,
                                   levels = c("Non-MSA", "MSA, Not Principle  City", "MSA, Principle Cit
                                   ordered = TRUE)),
    across(setdiff(categorical_vars, c("income_level", "age_bracket", "qualification", "census_msa")),
           ~ as.numeric(as.factor(.)))
  )

set.seed(123)
train_index <- createDataPartition(data_encoded$h1n1_vaccine, p = 0.7, list = FALSE)
train_data <- data_encoded[train_index, ]
test_data <- data_encoded[-train_index, ]
```

```
important_vars <- c("h1n1_vaccine", "census_msa", "income_level", "race", "qualification")
data_imputed$income_level <- factor(data_imputed$income_level,levels = unique(data_imputed$income_level
data_imputed$census_msa <- factor(data_imputed$census_msa,levels = unique(data_imputed$census_msa), labe
data_imputed$qualification <- factor(data_imputed$qualification,levels = unique(data_imputed$qualificati
plots <- lapply(important_vars, function(var) {
  ggplot(data_imputed, aes_string(x = var)) +
    geom_bar(fill = "skyblue", color = "black") +
    labs(title = paste(var), x = var, y = "Count") +
    theme_minimal() +
    theme(axis.text.x = element_text(angle = 45, hjust = 1),
          plot.title = element_text(size = 14, face = "bold"))
})
```

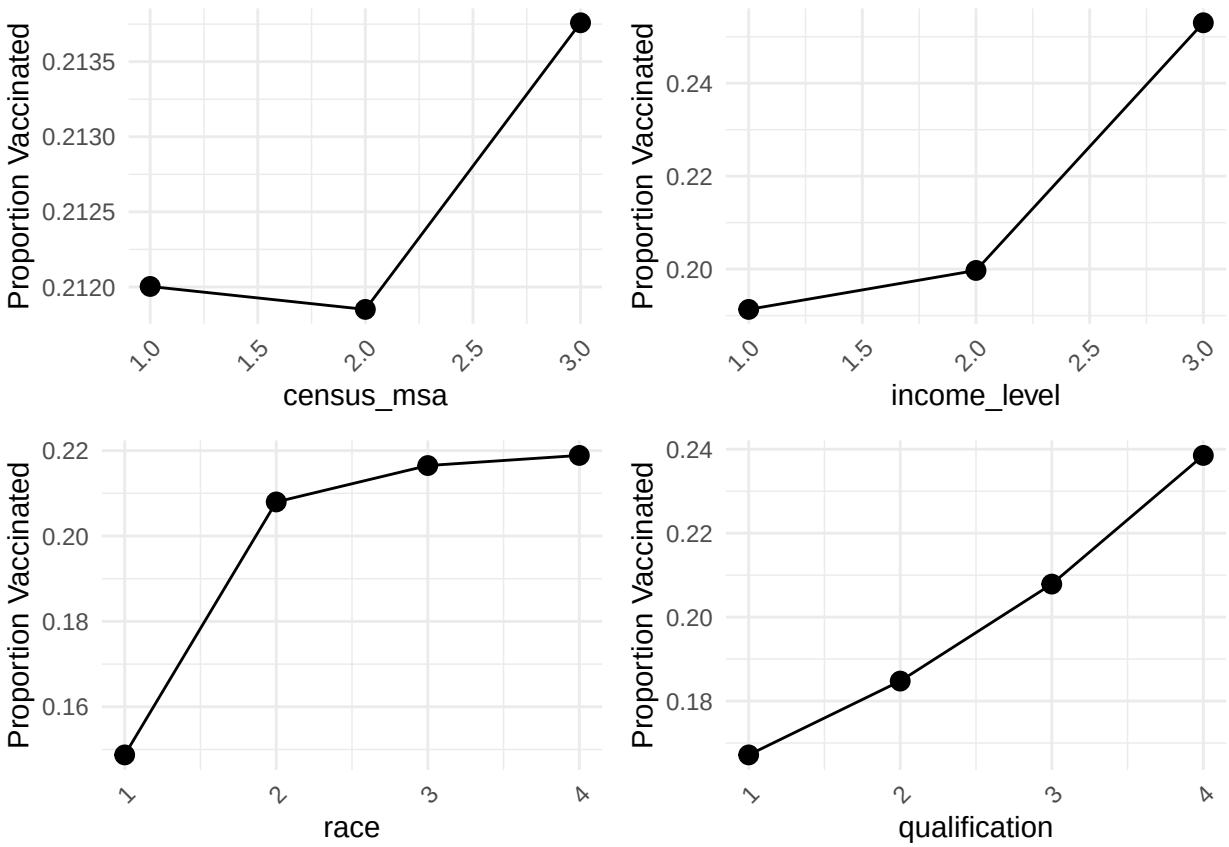
```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
```

```
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```r
grid.arrange(grobs = plots, ncol = 3, nrow = 2)
```




```r
exploratory_vars <- c("census_msa", "income_level", "race", "qualification")
plots <- lapply(exploratory_vars, function(var) {
  data_encoded %>%
    group_by(.data[[var]]) %>%
    summarize(mean_vaccine = mean(h1n1_vaccine, na.rm = TRUE)) %>%
    ggplot(aes(x = .data[[var]], y = mean_vaccine)) +
    geom_point(size = 3) +
    geom_line(group = 1) +
    labs(
      x = var,
      y = "Proportion Vaccinated"
    ) +
    theme_minimal() +
    theme(
      axis.text.x = element_text(angle = 45, hjust = 1),
      plot.title = element_text(size = 14, face = "bold")
    )
})

grid.arrange(grobs = plots, ncol = 2, nrow = 2)
```



The plots show how census_msa, income_level, race(unranked), and qualification influence H1N1 vaccination rates. Qualification displays the most linear and significant trend, with vaccination rates increasing across education levels. Income_level also shows an upward trend, while census_msa exhibit more moderate or non-linear patterns. The clear differences between education levels make qualification an ideal candidate for assigning distinct alphas in a Bayesian hierarchical structure.

feature selection & stan model for qualification

```r
library(glmnet)

x_train <- as.matrix(train_data %>% select(-h1n1_vaccine, -unique_id, -qualification))
x_test <- as.matrix(test_data %>% select(-h1n1_vaccine, -unique_id, -qualification))
y_train <- train_data$h1n1_vaccine

lasso_model <- cv.glmnet(
  x_train, y_train,
  alpha = 1,                    # L1 penalty for Lasso
  family = "binomial",          # Logistic regression
  type.measure = "auc",         # Use AUC for model evaluation
  nfolds = 5                    # Perform 5-fold cross-validation
)

selected_coefficients <- as.matrix(coef(lasso_model, s = "lambda.min"))
selected_features <- rownames(selected_coefficients)[selected_coefficients[, 1] != 0][-1]
```

```r
print("Selected Features:")
```

```
## [1] "Selected Features:"
```

```r
print(selected_features)
```

```
##  [1] "h1n1_worry"                "h1n1_awareness"
##  [3] "antiviral_medication"      "contact_avoidance"
##  [5] "bought_face_mask"          "avoid_large_gatherings"
##  [7] "reduced_outside_home_cont" "dr_recc_h1n1_vacc"
##  [9] "dr_recc_seasonal_vacc"     "chronic_medic_condition"
## [11] "cont_child_undr_6_mnths"   "is_health_worker"
## [13] "has_health_insur"          "is_h1n1_vacc_effective"
## [15] "is_h1n1_risky"             "sick_from_h1n1_vacc"
## [17] "is_seas_vacc_effective"    "is_seas_risky"
## [19] "sick_from_seas_vacc"       "age_bracket"
## [21] "race"                      "sex"
## [23] "income_level"              "marital_status"
## [25] "housing_status"            "employment"
## [27] "no_of_children"
```

```r
x_selected <- x_train[, selected_features]
x_test_selected <- x_test[, selected_features]

selected_variable_names <- selected_features
original_vars <- colnames(x_train)
removed_features <- setdiff(original_vars, selected_variable_names)
print("Removed Features:")
```

```
## [1] "Removed Features:"
```

```r
print(removed_features)
```

```
## [1] "wash_hands_frequently" "avoid_touch_face"      "census_msa"
## [4] "no_of_adults"
```

The lasso regression uses L1 penalty that forces some coefficients to become exactly zero, so effectively excluding those variables from the model. "wash_hands_frequently" is eliminated variables because they likely showed weak or no significant correlation with the likelihood of receiving the H1N1 vaccine compared to other variables.

```r
stan_data <- list(
  N = nrow(train_data),
  J = length(unique(train_data$qualification)),
  K = ncol(x_selected),
  y = y_train,
  x = x_selected,
  qualification = as.numeric(as.factor(train_data$qualification))
)

stan_model_qualification <- "
data {
  int<lower=0> N;   // Total number of rows in the dataset
  int<lower=0> J;   // Number of unique levels in the qualification variable
  int<lower=0> K;   // Number of selected predictors
  int<lower=0,upper=1> y[N];   // Target variable (whether vaccinated or not)
  matrix[N, K] x;   // Selected predictors for each individual
```

```
  int<lower=1,upper=J> qualification[N];   // Encoded education levels for each individual
}

parameters {
  vector[K] beta;   // Coefficients for predictors
  vector[J] alpha;   // Separate intercept for each education level
  real mu_alpha;    // Hyperparameter for the intercepts: Mean of the group-level intercepts
  real<lower=0> sigma_alpha;   // Also Hyperparameter for the intercepts: Standard deviation of group-le
}

model {
  alpha ~ normal(mu_alpha, sigma_alpha);   // Group-level Intercepts that follow a normal distribution
  beta ~ normal(0, 1);   // Informative normal prior for predictors
  mu_alpha ~ normal(0, 1);   // Prior for mean intercepts
  sigma_alpha ~ cauchy(0, 1.5);   // Prior for standard deviation of intercepts
  y ~ bernoulli_logit(alpha[qualification] + x * beta);   // Binary outcome y with the logistic regressi
}

generated quantities {
  vector[N] theta;   // Predicted probabilities
  vector[N] log_lik;   // Log-likelihood for each observation
  for (n in 1:N) {
    theta[n] = inv_logit(alpha[qualification[n]] + x[n] * beta);
    log_lik[n] = bernoulli_lpmf(y[n] | theta[n]);
  }
}
"

stan_model_qualification <- stan_model(model_code = stan_model_qualification)
```

```
## Trying to compile a simple C file
```

```
## Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
## using C compiler: 'Apple clang version 16.0.0 (clang-1600.0.26.4)'
## using SDK: ''
## clang -arch arm64 -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG   -I"/Library/Framew
## In file included from <built-in>:1:
## In file included from /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/StanHeade
## In file included from /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/RcppEigen/
## In file included from /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/RcppEigen/
## /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/RcppEigen/include/Eigen/src/Core
##   679 | #include <cmath>
##       |          ^~~~~~
## 1 error generated.
## make: *** [foo.o] Error 1
```

```
fit_final <- sampling(stan_model_qualification, data = stan_data, iter = 2000, chains = 4, seed = 123, c
```

```
print(fit_final, pars = c("beta", "alpha", "mu_alpha", "sigma_alpha"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##              mean se_mean   sd  2.5%   25%   50%   75% 97.5% n_eff Rhat
```

```
## beta[1]      -0.11    0.00 0.03 -0.16 -0.13 -0.11 -0.09 -0.05  5367    1
## beta[2]       0.15    0.00 0.04  0.07  0.13  0.15  0.18  0.23  4970    1
## beta[3]       0.17    0.00 0.10 -0.03  0.10  0.17  0.24  0.36  6919    1
## beta[4]      -0.06    0.00 0.05 -0.16 -0.10 -0.06 -0.03  0.04  5174    1
## beta[5]       0.32    0.00 0.08  0.16  0.26  0.32  0.37  0.48  5920    1
## beta[6]      -0.24    0.00 0.06 -0.35 -0.28 -0.24 -0.20 -0.13  4855    1
## beta[7]       0.00    0.00 0.06 -0.11 -0.04  0.00  0.03  0.11  4916    1
## beta[8]       2.05    0.00 0.06  1.93  2.01  2.05  2.10  2.18  3703    1
## beta[9]      -0.50    0.00 0.06 -0.62 -0.54 -0.50 -0.46 -0.38  3637    1
## beta[10]      0.06    0.00 0.05 -0.03  0.03  0.06  0.09  0.15  6068    1
## beta[11]      0.26    0.00 0.07  0.12  0.21  0.26  0.31  0.40  6545    1
## beta[12]      0.83    0.00 0.06  0.71  0.79  0.83  0.88  0.95  4425    1
## beta[13]      0.06    0.00 0.10 -0.12 -0.01  0.06  0.13  0.25  4144    1
## beta[14]      0.62    0.00 0.03  0.56  0.60  0.62  0.64  0.68  4056    1
## beta[15]      0.34    0.00 0.02  0.31  0.33  0.34  0.36  0.38  5012    1
## beta[16]     -0.01    0.00 0.02 -0.04 -0.02 -0.01  0.01  0.03  4429    1
## beta[17]      0.10    0.00 0.03  0.05  0.08  0.10  0.12  0.15  4610    1
## beta[18]      0.17    0.00 0.02  0.13  0.16  0.17  0.19  0.21  4736    1
## beta[19]     -0.08    0.00 0.02 -0.11 -0.09 -0.08 -0.06 -0.04  4551    1
## beta[20]      0.13    0.00 0.02  0.09  0.11  0.13  0.14  0.16  3667    1
## beta[21]      0.09    0.00 0.03  0.04  0.07  0.09  0.11  0.14  3510    1
## beta[22]      0.14    0.00 0.05  0.06  0.11  0.14  0.17  0.23  4435    1
## beta[23]      0.00    0.00 0.04 -0.09 -0.03  0.00  0.02  0.08  3506    1
## beta[24]     -0.16    0.00 0.05 -0.25 -0.19 -0.16 -0.13 -0.07  3575    1
## beta[25]     -0.03    0.00 0.06 -0.15 -0.07 -0.03  0.01  0.08  3079    1
## beta[26]      0.05    0.00 0.04 -0.03  0.02  0.05  0.07  0.13  4895    1
## beta[27]     -0.04    0.00 0.03 -0.09 -0.06 -0.04 -0.02  0.01  4111    1
## alpha[1]     -6.93    0.01 0.29 -7.50 -7.12 -6.94 -6.74 -6.36   874    1
## alpha[2]     -6.80    0.01 0.29 -7.34 -6.99 -6.80 -6.61 -6.23   865    1
## alpha[3]     -6.73    0.01 0.29 -7.28 -6.92 -6.73 -6.54 -6.16   883    1
## alpha[4]     -6.66    0.01 0.29 -7.21 -6.85 -6.66 -6.47 -6.08   856    1
## mu_alpha     -0.88    0.02 1.07 -2.99 -1.58 -0.87 -0.17  1.19  4273    1
## sigma_alpha   6.38    0.05 2.97  2.82  4.45  5.71  7.54 13.89  3200    1
##
## Samples were drawn using NUTS(diag_e) at Sun Dec  8 17:04:22 2024.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

feature selection & stan model for census_msa

```r
x_train_2 <- as.matrix(train_data %>% select(-h1n1_vaccine, -unique_id, -census_msa))
x_test_2 <- as.matrix(test_data %>% select(-h1n1_vaccine, -unique_id, -census_msa))

lasso_model_2 <- cv.glmnet(
  x_train_2, y_train,
  alpha = 1,                    # L1 penalty for Lasso
  family = "binomial",          # Logistic regression
  type.measure = "auc",         # Use AUC for model evaluation
  nfolds = 5                    # Perform 5-fold cross-validation
)

selected_coefficients_2 <- as.matrix(coef(lasso_model_2, s = "lambda.min"))
```

```r
selected_features_2 <- rownames(selected_coefficients_2)[selected_coefficients_2[, 1] != 0][-1]
print("Selected Features:")
```

```
## [1] "Selected Features:"
```

```r
print(selected_features_2)
```

```
##  [1] "h1n1_worry"              "h1n1_awareness"
##  [3] "antiviral_medication"    "contact_avoidance"
##  [5] "bought_face_mask"        "avoid_large_gatherings"
##  [7] "dr_recc_h1n1_vacc"       "dr_recc_seasonal_vacc"
##  [9] "chronic_medic_condition" "cont_child_undr_6_mnths"
## [11] "is_health_worker"        "has_health_insur"
## [13] "is_h1n1_vacc_effective"  "is_h1n1_risky"
## [15] "sick_from_h1n1_vacc"     "is_seas_vacc_effective"
## [17] "is_seas_risky"           "sick_from_seas_vacc"
## [19] "age_bracket"             "qualification"
## [21] "race"                    "sex"
## [23] "marital_status"          "housing_status"
## [25] "employment"              "no_of_children"
```

```r
x_selected_2 <- x_train_2[, selected_features_2]
x_test_selected_2 <- x_test_2[, selected_features_2]

original_vars_2 <- colnames(x_train_2)
removed_features_2 <- setdiff(original_vars_2, selected_features_2)
print("Removed Features:")
```

```
## [1] "Removed Features:"
```

```r
print(removed_features_2)
```

```
## [1] "wash_hands_frequently"     "reduced_outside_home_cont"
## [3] "avoid_touch_face"          "income_level"
## [5] "no_of_adults"
```

```r
stan_data_2 <- list(
  N = nrow(train_data),
  J = length(unique(train_data$census_msa)),
  K = ncol(x_selected_2),
  y = y_train,
  x = x_selected_2,
  census = as.numeric(as.factor(train_data$census_msa))
)

stan_model_census <- "
data {
  int<lower=0> N;
  int<lower=0> J;
  int<lower=0> K;
  int<lower=0,upper=1> y[N];
  matrix[N, K] x;
  int<lower=1,upper=J> census[N];   // census_msa categories
}

parameters {
```

```
  vector[K] beta;
  vector[J] alpha;   // Group-level intercepts for census_msa
  real mu_alpha;
  real<lower=0> sigma_alpha;
}

model {
  alpha ~ normal(mu_alpha, sigma_alpha);
  beta ~ normal(0, 1);
  mu_alpha ~ normal(0, 1);
  sigma_alpha ~ cauchy(0, 1.5);

  y ~ bernoulli_logit(alpha[census] + x * beta);
}

generated quantities {
  vector[N] theta;
  vector[N] log_lik;
  for (n in 1:N) {
    theta[n] = inv_logit(alpha[census[n]] + x[n] * beta);
    log_lik[n] = bernoulli_lpmf(y[n] | theta[n]);
  }
}
"

stan_model_census <- stan_model(model_code = stan_model_census)
```

```
## Trying to compile a simple C file
```

```
## Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
## using C compiler: ‘Apple clang version 16.0.0 (clang-1600.0.26.4)’
## using SDK: ‘’
## clang -arch arm64 -I"/Library/Frameworks/R.framework/Resources/include" -DNDEBUG   -I"/Library/Framew
## In file included from <built-in>:1:
## In file included from /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/StanHeade
## In file included from /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/RcppEigen/
## In file included from /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/RcppEigen/
## /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/library/RcppEigen/include/Eigen/src/Core
##   679 | #include <cmath>
##       |          ^~~~~~
## 1 error generated.
## make: *** [foo.o] Error 1
```

```
fit_final_2 <- sampling(stan_model_census, data = stan_data_2, iter = 2000, chains = 4, seed = 123, core
```

```
print(fit_final_2, pars = c("beta", "alpha", "mu_alpha", "sigma_alpha"))
```

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##              mean se_mean   sd  2.5%   25%   50%   75% 97.5% n_eff Rhat
## beta[1]     -0.11    0.00 0.03 -0.16 -0.13 -0.11 -0.09 -0.05  5666    1
## beta[2]      0.15    0.00 0.04  0.08  0.13  0.15  0.18  0.23  5687    1
## beta[3]      0.17    0.00 0.10 -0.03  0.10  0.17  0.23  0.36  6515    1
```

```
## beta[4]     -0.06    0.00 0.05 -0.16 -0.10 -0.06 -0.03  0.04  7407    1
## beta[5]      0.31    0.00 0.08  0.16  0.26  0.31  0.37  0.47  6740    1
## beta[6]     -0.24    0.00 0.05 -0.33 -0.27 -0.24 -0.21 -0.15  6305    1
## beta[7]      2.05    0.00 0.06  1.93  2.01  2.05  2.09  2.18  4651    1
## beta[8]     -0.50    0.00 0.06 -0.62 -0.54 -0.50 -0.46 -0.38  4561    1
## beta[9]      0.06    0.00 0.05 -0.03  0.03  0.06  0.09  0.16  6302    1
## beta[10]     0.26    0.00 0.07  0.11  0.21  0.26  0.31  0.41  6913    1
## beta[11]     0.83    0.00 0.06  0.71  0.79  0.83  0.88  0.96  6987    1
## beta[12]     0.06    0.00 0.10 -0.14  0.00  0.06  0.13  0.26  5251    1
## beta[13]     0.62    0.00 0.03  0.56  0.60  0.62  0.64  0.68  5198    1
## beta[14]     0.34    0.00 0.02  0.30  0.33  0.34  0.36  0.39  5481    1
## beta[15]    -0.01    0.00 0.02 -0.04 -0.02  0.00  0.01  0.03  4328    1
## beta[16]     0.10    0.00 0.03  0.05  0.08  0.10  0.12  0.16  6004    1
## beta[17]     0.17    0.00 0.02  0.14  0.16  0.17  0.19  0.21  5587    1
## beta[18]    -0.08    0.00 0.02 -0.12 -0.09 -0.08 -0.07 -0.04  4897    1
## beta[19]     0.13    0.00 0.02  0.09  0.11  0.13  0.14  0.16  4181    1
## beta[20]     0.08    0.00 0.02  0.03  0.06  0.08  0.10  0.13  4611    1
## beta[21]     0.09    0.00 0.03  0.04  0.07  0.09  0.11  0.14  4181    1
## beta[22]     0.14    0.00 0.05  0.06  0.11  0.14  0.17  0.23  5626    1
## beta[23]    -0.16    0.00 0.05 -0.25 -0.19 -0.16 -0.13 -0.07  5555    1
## beta[24]    -0.04    0.00 0.06 -0.15 -0.08 -0.04  0.00  0.08  4701    1
## beta[25]     0.05    0.00 0.04 -0.03  0.02  0.05  0.08  0.13  5282    1
## beta[26]    -0.04    0.00 0.03 -0.09 -0.06 -0.04 -0.02  0.01  4405    1
## alpha[1]    -6.97    0.01 0.27 -7.51 -7.14 -6.97 -6.79 -6.43  1349    1
## alpha[2]    -7.02    0.01 0.27 -7.56 -7.19 -7.02 -6.84 -6.49  1325    1
## alpha[3]    -6.97    0.01 0.27 -7.50 -7.14 -6.97 -6.78 -6.43  1340    1
## mu_alpha    -0.62    0.01 1.06 -2.74 -1.32 -0.59  0.10  1.46  5969    1
## sigma_alpha  7.01    0.07 3.66  2.98  4.66  6.09  8.17 17.10  3053    1
##
## Samples were drawn using NUTS(diag_e) at Sun Dec  8 17:08:51 2024.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```r
# Extract selected feature names from the feature selection process
selected_feature_names <- colnames(x_selected_2)

# Create a data frame mapping beta indices to predictor names
beta_mapping <- data.frame(
  Beta = paste0("beta[", seq_along(selected_feature_names), "]"),  # Beta indices
  Predictor = selected_feature_names                               # Corresponding predictors
)

# Print the mapping
beta_mapping
```

```
##        Beta               Predictor
## 1   beta[1]              h1n1_worry
## 2   beta[2]          h1n1_awareness
## 3   beta[3]    antiviral_medication
## 4   beta[4]       contact_avoidance
## 5   beta[5]        bought_face_mask
## 6   beta[6]  avoid_large_gatherings
## 7   beta[7]       dr_recc_h1n1_vacc
## 8   beta[8]   dr_recc_seasonal_vacc
```

```
## 9   beta[9] chronic_medic_condition
## 10 beta[10] cont_child_undr_6_mnths
## 11 beta[11]        is_health_worker
## 12 beta[12]        has_health_insur
## 13 beta[13]  is_h1n1_vacc_effective
## 14 beta[14]           is_h1n1_risky
## 15 beta[15]     sick_from_h1n1_vacc
## 16 beta[16]  is_seas_vacc_effective
## 17 beta[17]           is_seas_risky
## 18 beta[18]     sick_from_seas_vacc
## 19 beta[19]             age_bracket
## 20 beta[20]           qualification
## 21 beta[21]                    race
## 22 beta[22]                     sex
## 23 beta[23]          marital_status
## 24 beta[24]          housing_status
## 25 beta[25]              employment
## 26 beta[26]          no_of_children
```

Use of loo package for comparison of the two stan models

```r
library("loo")
```

```
## Warning: package 'loo' was built under R version 4.3.3

## This is loo version 2.8.0

## - Online documentation and vignettes at mc-stan.org/loo

## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use the 'cores' arg

##
## Attaching package: 'loo'

## The following object is masked from 'package:rstan':
##
##     loo
```

```r
# Log-likelihood matrix for the qualification model
log_lik_qualification <- extract_log_lik(fit_final, parameter_name = "log_lik")
loo_qualification <- loo(log_lik_qualification)

# Log-likelihood matrix for the census_msa model
log_lik_census <- extract_log_lik(fit_final_2, parameter_name = "log_lik")
loo_census <- loo(log_lik_census)

# compare two models
comparison <- loo_compare(loo_qualification, loo_census)
print(comparison)
```

```
##        elpd_diff se_diff
## model2  0.0       0.0
## model1 -1.9       1.4
```

Make predictions on test data using the posterior samples from the model of census_msa

```
# Posterior samples for alpha, the group-level intercepts
posterior_alpha <- extract(fit_final_2, pars = "alpha")$alpha

# Posterior samples for beta, the coefficients of the predictors
posterior_beta <- extract(fit_final_2, pars = "beta")$beta

stan_data_test <- list(
  N = nrow(test_data),   # Total number of test observations
  J = length(unique(train_data$census)),   # Number of unique census groups in the training data
  K = ncol(x_test_selected_2),   # Number of predictors included in the test matrix
  x = x_test_selected_2,   # The matrix of selected predictors for the test data
  census = as.numeric(as.factor(test_data$census_msa))   # Encoded group IDs for the census_msa variabl
)

# Compute the posterior linear predictor for each test observation
linear_predictors <- sapply(1:nrow(x_test_selected_2), function(n) {
  posterior_alpha[, stan_data_test$census[n]] +
    as.numeric(matrix(x_test_selected_2[n, ], nrow = 1) %*% t(posterior_beta))
})

# Convert the linear predictors into probabilities using the sigmoid function
predicted_probabilities <- colMeans(1 / (1 + exp(-linear_predictors)))
test_data$predicted_probabilities <- predicted_probabilities
```

We compared two models using Leave-One-Out Cross-Validation (LOO-CV) and generates predictions for the test data. Log-likelihood values for the models (fit_final with qualification and fit_final_2 with census_msa) are extracted and used to compute LOO-CV scores with the loo package, allowing for a comparison of their predictive accuracy. Posterior samples for group-level effects (alpha) and predictors (beta) are extracted from fit_final_2. For each test observation, linear predictors are computed by combining group-level intercepts with the test predictors. Predicted probabilities are calculated using the logistic function, averaged across posterior samples, and appended to the test dataset, enabling evaluation of the model's predictions.

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

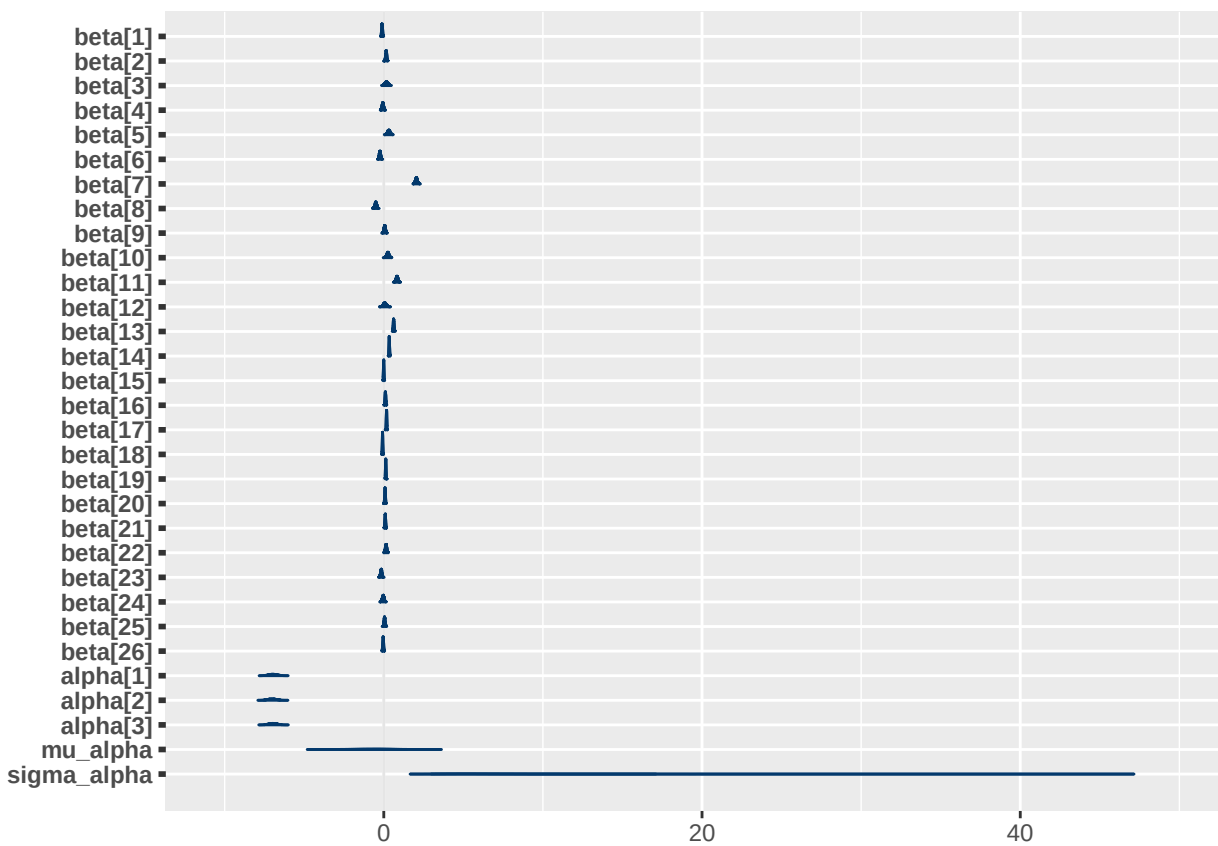
```
# Compute AUC
auc <- roc(test_data$h1n1_vaccine, test_data$predicted_probabilities)$auc
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```
cat("AUC:", auc, "\n")
```

```
## AUC: 0.8206099
```

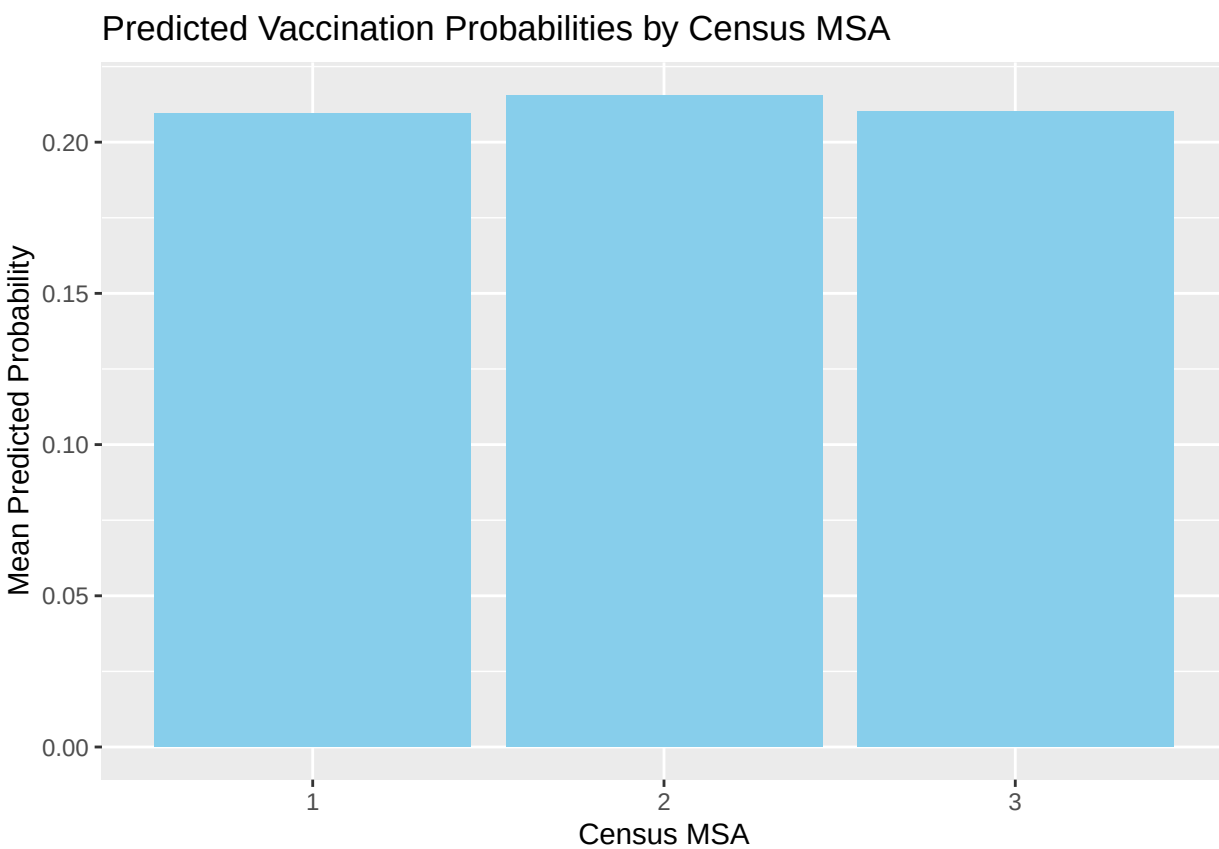
```r
library(bayesplot)

posterior_samples <- extract(fit_final_2)
all_samples <- cbind(
  as.matrix(posterior_samples$beta),
  as.matrix(posterior_samples$alpha),
  posterior_samples$mu_alpha,
  posterior_samples$sigma_alpha
)
colnames(all_samples) <- c(
  paste0("beta[", 1:ncol(posterior_samples$beta), "]"),
  paste0("alpha[", 1:ncol(posterior_samples$alpha), "]"),
  "mu_alpha", "sigma_alpha"
)

mcmc_areas(
  as.matrix(all_samples),
  pars = colnames(all_samples),
  prob = 0.95
)
```



The summary evaluates model performance using AUC and visualizes posterior predictions. AUC is computed with pROC to assess the model's ability to distinguish vaccinated from non-vaccinated individuals, with higher values indicating better performance. Posterior samples of theta (predicted probabilities) are summarized (mean, median, 95% credible intervals) and visualized for the first five parameters using mcmc_areas, highlighting the uncertainty in predictions.

additional 1: Summarize Predictions by Census Group

```
theta_samples <- extract(fit_final_2, pars = "theta")$theta

test_data$census_msa <- as.factor(test_data$census_msa)  # Ensure correct census labels

grouped_predictions <- aggregate(
  test_data$predicted_probabilities,
  by = list(census_msa = test_data$census_msa),
  FUN = mean
)
colnames(grouped_predictions) <- c("census_msa", "mean_probability")
print(grouped_predictions)
```

```
##   census_msa mean_probability
## 1          1        0.2093553
## 2          2        0.2155808
## 3          3        0.2102103
```

```
ggplot(grouped_predictions, aes(x = census_msa, y = mean_probability)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  labs(title = "Predicted Vaccination Probabilities by Census MSA",
       y = "Mean Predicted Probability",
       x = "Census MSA")
```